

Fact-based News Verification System

A Hybrid System with BERT Classification, GNews Evidence Retrieval, and
Fine-tuned NLI Verification

Name	Student ID	Email
Sun Yuchen	A0326248B	e1538079@u.nus.edu
Zhao Jiahui	A0329852U	e1554179@u.nus.edu
Zhang Yuxuan	A0285664N	e1216649@u.nus.edu

Abstract: This project develops a fact-based news verification system for online news articles. Instead of relying only on article-level fake/real classification, the system combines a fine-tuned BERT baseline classifier with sentence-level evidence retrieval and a fine-tuned DeBERTa-based Natural Language Inference verifier. The input article is cleaned and split into sentence-level verification units. Each sentence is used to retrieve external evidence from GNews, and the NLI model determines whether the evidence supports, refutes, or provides insufficient information for the sentence. The sentence-level results are aggregated into an article-level verdict. The system also includes sentiment analysis, LDA-based topic analysis, and a Streamlit chat interface to improve interpretability and user interaction.

Keywords: BERT Classification, GNews Retrieval, Natural Language Inference, Evidence Verification, Sentiment Analysis, Topic Modelling, Streamlit Chat

Contents

1	Business Problem and Objectives	1
1.1	Problem Context	1
1.2	Business Value	1
1.3	Project Objectives	1
1.4	Scope	2
2	Data and Evidence Resources	2
3	System Design and Implementation	3
4	Test Results	4
5	Limitations and Future Work	7
6	Conclusion	7
7	References	8
7.1	Libraries and Frameworks	8
7.2	Models, APIs, and Data Sources	8
8	Individual Contributions	9
9	GenAI / LLM Declaration	9

1 Business Problem and Objectives

1.1 Problem Context

Online news and social-media links move faster than manual verification workflows. A reader may encounter a politically sensitive story, a health claim, or a market-related rumour within minutes of publication, but checking whether the factual statements are supported by external reporting usually requires searching multiple sources, comparing wording, and deciding whether the evidence actually supports the claim. This creates a practical business problem: users need fast screening, but they also need a reason to trust the screening result.

A conventional fake/real classifier only solves part of this problem. It can produce a quick label, but it normally cannot show which sentence is wrong, what external evidence was used, or whether the article contains a mixture of true, false, and unverifiable statements. Such classifiers can also learn topic, source, or writing-style patterns instead of factual correctness. This is especially risky in real news workflows, where an article can be written in a professional style while still including outdated numbers, missing context, or claims that are contradicted by newer reporting.

1.2 Business Value

The business value of the project is to reduce the effort required for first-pass news verification. Instead of asking a user to manually search every sentence, the system provides an initial article-level warning, retrieves external news evidence, and highlights which statements are supported, refuted, or not sufficiently evidenced. This makes the output more actionable than a single fake/real probability: the user can inspect the evidence trail, decide which sentences deserve attention, and use the result as a starting point for deeper human review.

The system is relevant to several user groups. General readers can use it to check suspicious articles before sharing them. Students and researchers can use it as an information-literacy tool that demonstrates the difference between style-based classification and evidence-based verification. Journalists, editors, and fact-checking teams can use it as a triage assistant that surfaces potentially unsupported statements before a human reviewer spends time on the full article. For these users, the value is not that the system replaces expert judgement; the value is that it makes verification faster, more transparent, and more reproducible.

1.3 Project Objectives

The main objective is to build an explainable news verification assistant that combines article-level screening with sentence-level evidence checking. Given a news article, the system should clean the text, estimate an initial BERT-based fake/real prior, split the article into verification units, retrieve external evidence from GNews, verify each sentence-evidence pair with a fine-tuned NLI model, and aggregate the sentence-level results into a final article-level verdict.

The second objective is interpretability. The system should not only output **Likely True**, **Likely False**, or **Unverified**; it should also show the retrieved evidence and the sentence-level labels that contributed to the verdict. This helps users understand why the model reached a decision and where uncertainty remains. Sentiment analysis, topic analysis, and the Streamlit chat interface are included for this reason: they make the system easier to inspect and discuss, but they do not replace the core evidence-verification pipeline.

The third objective is practical deployability within the project scope. The system should run as an interactive Streamlit application, use saved model artefacts for repeatable inference, keep configuration values in one place, and separate the main runtime path from auxiliary modules such

as claim extraction and topic exploration. This makes the implementation easier to demonstrate, evaluate, and extend.

1.4 Scope

The project focuses on English text-based news verification. In scope are article or headline-style inputs, BERT-based baseline classification, sentence-level GNews retrieval, evidence reranking, DeBERTa-NLI verification, article-level aggregation, sentiment display, topic exploration, and a rule-based chat interface over the verification workflow. The system is designed for first-pass screening and evidence inspection, not for issuing legally or journalistically final truth judgements. Several tasks are outside the current scope. The system does not verify images, videos, audio, or deepfakes. It does not perform full multi-hop investigation across long documents, and it does not guarantee that GNews contains the most complete or authoritative evidence for every claim. It also does not replace professional editorial review. These limits define the system’s role: it is a transparent decision-support tool that helps users identify which statements are worth checking further.

2 Data and Evidence Resources

The project uses different data sources for different parts of the pipeline. Local fake/real news data support the baseline classifier and topic exploration, an adapted factual-consistency NLI dataset supports the verifier, a small claim-extraction dataset supports an auxiliary extractor, and GNews provides live evidence at inference time. This separation is important because the system is not trained on GNews results directly: the learned models are trained offline, while GNews supplies current external evidence during runtime.

For the baseline classifier, the repository uses four FakeNewsNet-style CSV files. These records are mainly title-level rather than full article bodies, so the BERT classifier is best understood as a fast article-level prior instead of a complete factual verifier.

Table 1: Local fake/real datasets used for baseline classification

File	Rows	Label	Role
<code>gossipcop_fake.csv</code>	5,323	Fake	Title-level fake-news examples for BERT training.
<code>gossipcop_real.csv</code>	16,817	Real	Title-level real-news examples for BERT training.
<code>politifact_fake.csv</code>	432	Fake	Additional fake-news titles for domain diversity.
<code>politifact_real.csv</code>	624	Real	Additional real-news titles for domain diversity.
Total	23,196	–	Local baseline data with 17,441 real and 5,755 fake rows.

The codebase also contains LIAR-style TSV loading support, but the final reported baseline model is the saved BERT classifier trained on the local GossipCop and PolitiFact split. For the verifier, the project uses an adapted factual-consistency NLI dataset stored under `data/nli_dataset/`. The records are derived from VitaminC-style premise-hypothesis pairs and contain 39,000 balanced three-way samples: 30,000 for training, 4,500 for validation, and 4,500 for testing. Each example pairs an evidence passage with a claim-like hypothesis, which makes the data closer to the deployed verification task than binary fake/real classification. The repository includes the final NLI dataset and saved fine-tuned verifier artefacts under `models/claim_verifier/final/`.

Table 2: Adapted factual-consistency NLI dataset used for verifier fine-tuning

Split	Samples	Class Balance	Role
<code>real_nli_train.jsonl</code>	30,000	Balanced 3-way	Fine-tuning the DeBERTa verifier.
<code>real_nli_valid.jsonl</code>	4,500	Balanced 3-way	Model selection and threshold tuning.
<code>real_nli_test.jsonl</code>	4,500	Balanced 3-way	Final held-out evaluation.
Total	39,000	13,000 per class	Balanced factual-consistency corpus for verifier fine-tuning.

The NLI records use the standard premise-hypothesis format: `premise` stores the evidence passage, `hypothesis` stores the sentence or factual statement to verify, and `label_id` maps contradiction, neutral, and entailment to refuted, NEI, and supported at inference time. The repository also contains `data/claim_extraction_dataset.json` for the auxiliary Flan-T5 claim extractor, although the deployed application defaults to sentence splitting because preserving the original sentence wording produced more reliable GNews search queries. Finally, the same GossipCop and PolitiFact data are reused for LDA topic exploration, where labels are used for analysis rather than for the final verification decision.

3 System Design and Implementation

The final system is an evidence-based verification pipeline rather than a simple fake-news classifier. The BERT classifier remains useful, but it serves as an article-level prior alongside sentence-level retrieval and NLI verification.

The end-to-end flow of the system is shown below:

```

Article Input → Text Cleaning → BERT Baseline Classifier
→ Sentence Splitting → GNews Retrieval → Evidence Reranking
→ Fine-tuned DeBERTa-NLI Verifier → Article-level Aggregation
→ Final Verdict + Evidence + Sentiment + Topic + Chat

```

This architecture balances speed and interpretability. The BERT classifier gives an immediate article-level signal, while the retrieval and NLI stages explain which specific factual statements are supported, contradicted, or still unverified. Before classification and retrieval, the article is lightly cleaned: URLs and social-media artefacts are removed, whitespace is normalized, and the original wording is otherwise preserved. The cleaned article is passed to the fine-tuned `bert-base-uncased` classifier, whose output is treated as a prior rather than a hard truth label because the training data are mostly headline-level examples.

The evidence path then splits the article into sentence-level search units. The codebase contains a trained Flan-T5 claim extractor, but the Streamlit application defaults to sentence splitting because aggressive claim rewriting can remove context and make GNews queries less reliable. Each sentence is sanitized into an API-safe query, submitted to GNews, and matched against returned titles, descriptions, and content snippets. The retriever reranks results using lexical overlap, entities, numbers, source relevance, and source credibility, so reputable outlets contribute more strongly than weak or noisy sources.

The central semantic component is the fine-tuned DeBERTa-NLI verifier. It is initialized from `MoritzLaurer/deberta-v3-base-mnli-fever-anli` and fine-tuned on the 39K-sample factual-

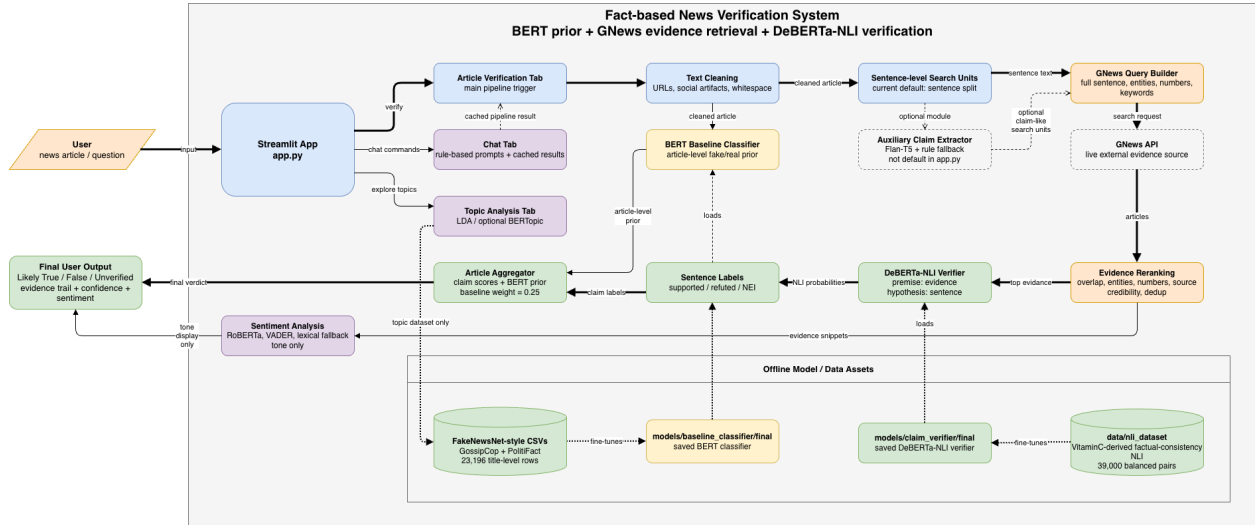


Figure 1: System structure and runtime workflow of the fact-based news verification system.

consistency NLI dataset in Table 2. At runtime, the retrieved evidence is used as the premise and the article sentence is used as the hypothesis. Entailment becomes **supported**, contradiction becomes **refuted**, and neutral becomes **NEI**. Long evidence snippets are processed in chunks, and the system selects the strongest credibility-weighted NLI signal rather than averaging across irrelevant passages.

The final verdict is produced by combining sentence-level verification labels with the BERT prior. Many supported sentences push the article toward **Likely True**; many refuted sentences push it toward **Likely False**; weak evidence or mostly neutral results lead to **Unverified**. Sentiment, topic analysis, and chat interaction are kept outside the core truth decision. Sentiment uses `cardiffnlp/twitter-roberta-base-sentiment-latest` with VADER and keyword fallbacks, topic analysis uses LDA with optional BERTopic, and the Streamlit chat tab is a rule-based interface that calls existing pipeline functions and caches results. These modules improve interpretability and usability without changing the factual verdict.

The final implementation is organized around the modules in Table 3. Runtime thresholds and scoring weights are centralized in `pyproject.toml` and can be overridden through environment variables, which makes the verification pipeline easier to tune without rewriting code.

4 Test Results

The final implementation includes saved trained models and reported evaluation results, so this section focuses on the two learned components that drive the runtime system: the BERT baseline classifier and the DeBERTa-NLI verifier. The baseline BERT classifier was evaluated on a 20% held-out split with 4,640 samples. It achieves accuracy 0.8752, binary F1 0.9189, and macro-F1 0.8245. The confusion matrix in Table 4 shows that the model is a useful article-level screening signal, but it also confirms the expected class-imbalance weakness: 368 of 1,151 fake examples are misclassified as real. This reinforces the system design choice to treat BERT as a prior rather than as the sole truth-verification mechanism.

The benefit of fine-tuning is clearer when the final BERT model is compared with weaker baselines. As shown in Table 5, the majority classifier obtains a deceptively high binary F1 because the dataset

Table 3: Main implementation modules and configuration files

Module or File	Responsibility
<code>app.py</code>	Streamlit interface, multi-tab workflow, chat interaction, and cached end-to-end orchestration.
<code>src/retriever.py</code>	GNews querying, retrieval-unit search, source credibility scoring, deduplication, and evidence reranking.
<code>src/verifier.py</code>	Fine-tuned DeBERTa NLI verification with chunked evidence processing, credibility weighting, and strongest-signal selection.
<code>src/claim_extractor.py</code>	Flan-T5 claim extraction with a rule-based fallback; retained as an auxiliary module.
<code>src/sentiment_analyser.py</code>	Media-tone analysis using RoBERTa sentiment with VADER and keyword fallbacks.
<code>src/topic_analysis_page.py</code>	LDA topic exploration, optional BERTopic integration, dataset explorer, and article-topic inference.
<code>models/baseline_classifier/</code>	Saved BERT baseline classifier artefacts used for article-level screening.
<code>models/claim_verifier/</code>	Saved fine-tuned DeBERTa verifier artefacts and intermediate checkpoints.
<code>pyproject.toml</code>	Central configuration for retrieval weights, verifier thresholds, and other overridable system parameters.

Table 4: Confusion matrix for the BERT baseline classifier

Actual / Predicted	Pred Fake	Pred Real
Actual Fake	783	368
Actual Real	211	3,278

contains many more real than fake examples. Macro-F1 gives the more reliable view: fine-tuning raises macro-F1 from 0.4802 for the untrained BERT head to 0.8245, and fake-news recall rises from 9.9% to 68.0%. The model is therefore strong enough to provide a prior, but not strong enough to replace evidence-based verification.

Table 5: Baseline classifier comparison on the 4,640-sample held-out test set

Model	Accuracy	P (binary)	R (binary)	F1 (binary)	F1 (macro)
Majority baseline (always Real)	0.7519	0.7519	1.0000	0.8584	0.4292
Original BERT (untrained head)	0.7013	0.7517	0.9000	0.8192	0.4802
Fine-tuned BERT (3 epochs)	0.8752	0.8991	0.9395	0.9189	0.8245

The final verifier was fine-tuned from `MoritzLaurer/deberta-v3-base-mnli-fever-anli` on the balanced factual-consistency NLI dataset in Table 2. Evaluation was performed on the 4,500-sample held-out test set. Earlier heuristic matching logic was useful during prototype development, but the deployed verifier is the fine-tuned NLI model reported here. It reaches accuracy 0.8644, macro precision 0.8649, macro recall 0.8644, and macro-F1 0.8643. Entailment is the strongest class, while contradiction and neutral remain slightly harder because retrieved news evidence often contains partial overlap or incomplete context. Table 6 shows that fine-tuning improves macro-F1 by 18.36 percentage points over the original pretrained checkpoint.

Table 6: Original versus fine-tuned NLI verifier on `real_nli_test.jsonl`

Metric	Original DeBERTa-v3	Fine-tuned NLI	Δ
Accuracy	0.6816	0.8644	+0.1829
Precision (macro)	0.7200	0.8649	+0.1450
Recall (macro)	0.6816	0.8644	+0.1829
F1 (macro)	0.6807	0.8643	+0.1836

The most operationally important NLI gain is contradiction recall, which rises from 52.13% to 81.87%. Missed contradictions would cause the system to overlook refuting evidence and produce overly optimistic support labels, so this improvement directly supports the evidence-verification goal. Across the two learned components, the pattern is consistent: pretrained language models provide useful initialization, but task-specific fine-tuning is what makes the system reliable enough for practical screening.

A complete evaluation should also include one worked example showing how the system behaves on a real article. Because final screenshots and demo evidence will be added separately, Table 7 is presented as an illustrative template for documenting sentence-level evidence, predicted labels, and the type of improvement expected from the NLI verifier.

This kind of case study is important because it demonstrates not only whether the final verdict is plausible, but also whether the evidence trail shown to the user is understandable and trustworthy. In particular, the NLI verifier is more reliable than simple overlap matching when numeric values or partially related phrases appear in the evidence but do not actually contradict the claim.

Table 7: Illustrative end-to-end case-study template

Sentence	Top Evidence	Target Label	Comment
Brockman disclosed financial ties and a nearly \$30B stake.	Reuters	Supported	Strong evidence match for the financial-interest statement.
Musk filed a lawsuit over the for-profit versus nonprofit issue.	Economic Times	Supported	Retrieved article directly addresses the litigation context.
Brockman’s independence may be compromised by startup investments.	Rappler	Supported	Evidence is related and provides supporting discussion rather than exact lexical overlap.
Stakes in startups and Altman’s family fund raise conflict-of-interest concerns.	Economic Times	Supported	Useful example of multi-entity matching across one sentence.
The trial and ChatGPT launch happened in 2022.	MarketScreener	NEI → Supported	Heuristic matching can overreact to unrelated numbers; NLI is intended to reduce this error.

Even after the NLI upgrade, the system still has several realistic error sources. GNews may return related but insufficient evidence, returned snippets can be truncated, long sentences may contain multiple subclaims, and a single wrongly refuted sentence can disproportionately influence the article-level aggregation. These limitations are expected in practical evidence-based verification and are addressed further in the next section.

5 Limitations and Future Work

The current system is more transparent than a pure article-level classifier, but it still depends heavily on retrieval quality and dataset fit. If the retrieved news snippets are incomplete or weakly related, even a strong NLI model cannot produce reliable verification labels. In addition, the baseline fake/real data are title-heavy, so they are better suited to screening than to deep factual reasoning. Future work should focus on four areas. First, the team should expand the NLI training set with manually validated news claim-evidence pairs and a cleaner manual validation split. Second, the retrieval stage can be improved with better sentence selection, reranking, and source filtering. Third, the aggregation rule can be calibrated with confidence weighting rather than simple counting. Fourth, the system can be extended to support multi-hop evidence and longer article-body verification beyond sentence-level snippets.

6 Conclusion

The final system demonstrates a hybrid approach to news verification. The BERT classifier provides a fast article-level fake/real prior, while sentence-level GNews retrieval and the fine-tuned NLI verifier provide evidence-based semantic verification. Sentiment analysis, LDA topic modelling, and

the chat interface improve interpretability and usability.

Compared with a pure fake-news classifier, this design is more transparent because it shows which sentences are supported, refuted, or still unverified and which external evidence was used. At the same time, the system remains limited by retrieval quality, snippet completeness, label-mapping noise, and the final accuracy of the NLI verifier. Even with those limitations, the project shows why evidence-based verification is a more convincing direction than relying only on article-level style classification.

7 References

7.1 Libraries and Frameworks

1. **Streamlit**. Used for the interactive application interface. Official documentation: Streamlit documentation. Chat component reference: Streamlit chat API reference. Public source code: streamlit/streamlit.
2. **PyTorch**. Used as the deep learning runtime for BERT and DeBERTa training and inference. Official documentation: PyTorch documentation. Public source code: pytorch/pytorch.
3. **Hugging Face Transformers and Datasets**. Used for model loading, tokenization, fine-tuning, and dataset preparation. Official documentation: Transformers documentation and Datasets documentation. Public source code: huggingface/transformers and huggingface/datasets.
4. **scikit-learn**. Used for train-test splitting, evaluation metrics, **CountVectorizer**, and LDA topic modelling. Official user guide: scikit-learn user guide. LDA reference: LatentDirichletAllocation documentation. Public source code: scikit-learn/scikit-learn.
5. **pandas, NumPy, and Requests**. Used for CSV processing, numerical operations, and API calls. Official documentation: pandas, NumPy, and Requests.
6. **NLTK / VADER**. Used as a fallback sentiment analyser when the transformer sentiment model is unavailable. Official documentation: NLTK documentation. VADER reference: VADER repository.
7. **BERTopic**. Optional topic-modelling extension used when the environment supports it. Official documentation: BERTopic documentation. Public source code: MaartenGr/BERTopic.

7.2 Models, APIs, and Data Sources

1. **bert-base-uncased**. Pretrained encoder used for the baseline fake/real classifier. Model card: Hugging Face model card.
2. **DeBERTa / DeBERTa-v3**. Backbone used for semantic verification through NLI fine-tuning. Model resources: DeBERTa-v3 base model card.
3. **MoritzLaurer/deberta-v3-base-mnli-fever-anli**. Practical starting checkpoint for NLI-based verification. Model card: Hugging Face model card.
4. **News-related NLI or stance-verification data**. Used to build claim-evidence-label training pairs for the verifier. One public reference is the Fake News Challenge Stage 1 dataset: FNC-1 repository. Stance labels require mapping and manual inspection before they can be used as NLI-style labels.
5. **GNews API**. Live evidence source used at inference time for sentence-level retrieval. Official documentation: GNews API documentation.
6. **cardiffnlp/twitter-roberta-base-sentiment-latest**. Sentiment model used as an interpretability feature. Model card: Hugging Face model card.
7. **Flan-T5**. Backbone used for the auxiliary claim extraction module. A practical reference

checkpoint is: google/flan-t5-base model card.

8. **FakeNewsNet-style GossipCop and PolitiFact data.** Local fake/real datasets used for the baseline classifier and topic analysis. Public repository reference: KaiDMML/FakeNewsNet.

8 Individual Contributions

Table 8: Planned contribution summary for the final report

Team Member	Main Responsibility	Contribution Details
Sun Yuchen	Dataset preparation, base-line BERT training, evaluation, and topic analysis	Prepared the fake/real datasets, organized the baseline training workflow, documented classifier evaluation, and supported LDA-based dataset exploration.
Zhao Jiahui	GNews retrieval, sentence-level pipeline, Streamlit app, and chat interaction	Implemented retrieval logic, sentence-level verification flow, evidence display, and the user-facing interface for verification and chat-style interaction.
Zhang Yuxuan	NLI dataset construction, DeBERTa-NLI fine-tuning, sentiment module, and verifier evaluation	Prepared claim-evidence training pairs, fine-tuned the NLI verifier, integrated sentiment analysis, and designed the verifier comparison metrics.
Shared	Report writing, demo preparation, and error analysis	All team members contributed to report revision, final presentation materials, discussion of limitations, and interpretation of system results.

9 GenAI / LLM Declaration

Generative AI tools were used to assist with debugging, report structuring, explanation refinement, and code review. The team manually reviewed, modified, and validated the generated suggestions before including them in the project. No confidential or private user data was submitted. The final system design, implementation decisions, experiments, and conclusions remain the responsibility of the project team.